**ADVANCED PROGRAMMING LANGUAGES**

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

# Challenge Set: Type Parameterization and Bounded Polymorphism

## 1. Diagnosing variance and bound

For each of teh following definitions determine whether or not the definition compiles.

```
// (a)
trait Container[+A] { def add(x: A): Container[A] }
// (b)
class Cell[+A](var content: A):
// (c)
def maxOf[A <: Comparable[A]](xs: List[A]): A
// (d)
def firstSeq[A](xs: Set[A]): Seq[A] = xs
// (e)
trait Stack[+A]:
  def push[B >: A](x: B): Stack[B]
  def peek: A
```

## 2. Design with F-bounds

Design a trait `Mergeable[A]` such that any type `A` extending `Mergeable[A]` provides a method `merge(that: A): A` for combining two values of the same concrete type. Then:

1. Implement two case classes that extend `Mergeable` : a `Counter(value: Int)` that merges by summing, and a `Log(entries: List[String])` that merges by concatenating.
2. Write a polymorphic function `mergeAll[A <: Mergeable[A]](xs: List[A]): Option[A]` that reduces a list using `merge` , returning `None` for the empty list.
3. Show — by attempting it and observing what the compiler says — what goes wrong if you drop the recursive bound and write `trait Mergeable` with `def merge(that: Mergeable): Mergeable` . Identify *which* concrete operation is no longer expressible.

# 3. A sorted collection hierarchy

Design a small parameterized hierarchy of immutable ordered collections:

- A top-level trait `Ordered[+A]` exposing only `def head: A` and `def tail: Ordered[A]`, sufficient to read elements in order.
- A subtrait `Insertable[A]` extending `Ordered[A]` that adds `def insert(x: A): Insertable[A]`. Decide the variance carefully, and justify it.
- Two concrete implementations: `SortedList[A]` (linked-list backing) and `SortedSet[A]` (binary search tree backing), each requiring `A` to admit an ordering.

Then answer:

1. Where did you need an upper bound, and what operations did the bound enable?
2. Why could `Ordered` be made covariant but `Insertable` could not? Or, if you found a way to keep `Insertable` covariant, what trick did you use?
   (Hint: lower bounds on method type parameters, like `Stack.push` in Challenge 1.)
3. Does `SortedList[A] <: SortedSet[A]` hold? Should it? Justify your answer in terms of the operations and their semantics, not just the declared hierarchy.